

插件初探

1. 导入语句
2. 装饰器
3. 类定义和继承
4. 异步编程
5. 类型注解
6. 字符串格式化
7. 方法重写
8. 文档字符串
9. 生成器

0. 注释方式

```
# 用于单行注释
```

```
"""
```

```
这是一个多行注释
```

```
虽然Python没有专门的多行注释语法
```

```
但三引号字符串常被用作多行注释
```

```
因为它会被解释器忽略（如果没有赋值给变量）
```

```
"""
```

1. 导入语法

```
from astrbot.api.event import filter, AstrMessageEvent,
MessageEventResult
```

```
from astrbot.api.star import Context, Star, register
```

```
from astrbot.api import logger
```

不同编程语言的“导入（Import）”机制，表面上是代码复用的手段，底层其实折射了该语言对**工程化、命名空间管理以及运行效率**的不同理解

C++

C++ 传统的 `#include` 并不是真正的“导入”，而是**预处理器指令**

语法特征： `#include <iostream>`

机制： 预处理器会直接把头文件的**全部内容**“复制、粘贴”到当前位置

反映特性：

- **零开销抽象：** 编译器不增加额外负担，但会导致编译时间极长（重复包含同一个头文件）。
- **脆弱的封装：** 需要依赖 `header guards` (`#ifndef`) 来防止重复定义



你一定写过 `#include <stdio.h>`。

这时候，编译器其实还没开始工作。真正工作的是**预处理器**

(Preprocessor)

想象你在写一篇论文（你的 `.c` 或 `.cpp` 文件），你需要引用一段别人的名言（头文件 `.h`）

C/C++ 的做法（`#include`）： 预处理器是个勤恳但不太聪明的抄写员。当它看到 `#include <stdio.h>` 时，它会把 `stdio.h` 文件里几千行的代码，一行不差地复制，直接覆盖掉 `#include` 这一行

“零开销抽象”的代价： C++ 为了让你在**运行程序**时非常快（零开销），它选择在**编译阶段**把所有东西都展开、复制、优化好。它把痛苦留给了编译过程，把快乐留给了运行过程

```
#include "astrbot/api/event.h"
#include "astrbot/api/star.h"
// 通常需要 using namespace std; 或使用 std::
```

python

Python 的 `import` 是**运行时执行**的。当你导入一个模块时，Python 解释器会真的去执行那个文件,而不是像C/C++一样全部复制

Python 特性：

- 使用 `import` 或 `from...import` 语句导入模块
- 支持模块化编程，避免命名冲突
- 动态导入，运行时解析

```
from astrbot.api.event import filter, AstrMessageEvent,
MessageEventResult
```

```
from astrbot.api.star import Context, Star, register
```

```
from astrbot.api import logger
```

两种导入方式：

1. `import` 语句

```
import module
```

```
import module1, module2
```

```
import module as alias
```

```
import module1 as alias1, module2 as alias2
```

我们把 `module` 称为**命名空间**

所谓命名空间是一个从**名称到对象**的映射

访问方式：必须通过 `module.name`（点号操作符）来访问其中的内容
这最安全，因为模块名起到了前缀的作用，保护了你当前的命名空间不被“污染”

模块是“独立的抽屉”

每个 `.py` 文件就是一个模块，它拥有自己独立的**全局命名空间**

2. `** from module import name` 语句

直接将抽屉里的东西拿出来，放在当前的桌子上，因此可以直接使用**name**即可，但是如果当前环境已经有同名变量就会发生覆盖。

从 `astrbot.api.event` 这个“大仓库”里，只挑出这三个具体的“零件”放到你当前的桌面上

你现在可以直接写 `filter` 或 `AstrMessageEvent`。它们被直接绑定到了你当前模块的**全局作用域**中

从最开始的导入，我们就可以深刻的感受到python的底层理念：模块即对象

在 Python 看来，`astrbot.api.event` 本身就是一个 **Module 对象**

物理上：它是磁盘上的一个 `.py` 文件或一个包含 `__init__.py` 的文件夹

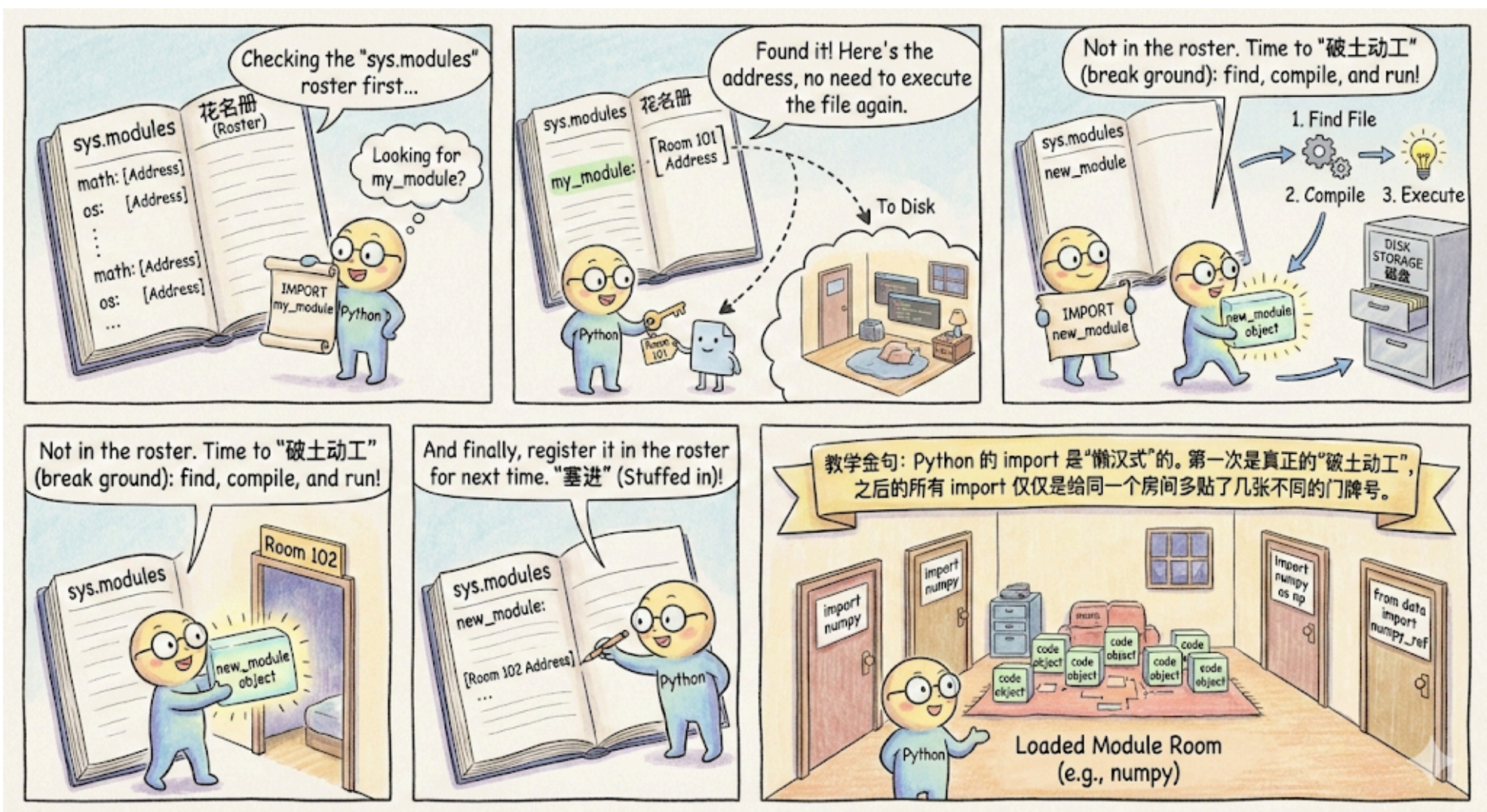
逻辑上：它是一个容器（即命名空间），用一个**字典**（`__dict__`）维护着所有的变量名和它们的内存地址

导入的本质：导入不是“复制代码”，而是“**建立引用**”。当你 `import` 时，Python 只是在你的命名空间里增加了一个指针，指向了目标模块里的对象

当你多次 `import astrbot` 时，Python **并不会**执行该文件多次

Python 在第一次导入时，会执行代码并把生成的模块对象存入 `sys.modules` 这个全局字典里。后续的 `import` 只是在你的局部空间加了个“快捷方式”指向这个现成的对象

底层逻辑：`sys.modules` **字典**



```
from astrbot.api.event import filter
```

警告: Python 有个内置函数也叫 `filter` (用于过滤序列)

当你这样导入后, 你插件里的 `filter` 就不再是 Python 原生的那个过滤函数, 而是 AstrBot 的事件过滤器了

亲自尝试:

```
import sys

import astrbot.api

# 实验: 多次导入同一个模块
import astrbot.api as api_alias

print(id(astrbot.api))

print(id(api_alias))

print(id(sys.modules['astrbot.api']))
```

输出：

```
PS E:\astrbot> E:/astrbot/.venv/Scripts/python.exe e:\astrbot\shiro\data\plugins\python_test.py
2015724860048
2015724860048
2015724860048
```

地址完全相同，也就是上图的房间一样，但是别名不一样。

Go

Go与C++类似，也是编译器决定的，但是比C++更聪明。

它不会盲目的复制，而是读取编译后的二进制导出文件 `.a` 文件

核心特性：强制导出规则。Go 规定：首字母大写的变量/函数才能被外部导入（Public），小写的只能在包内使用（Private）。

命名空间：Go 强制要求通过包名访问（如 `fmt.Println`），**不允许**类似 Python 的 `from fmt import Println` 这种“污染桌面”的操作。这体现了 Go 极端的**工程严谨性**

Go 抛弃了预处理器（Preprocessor）。Go 的编译器直接处理语法树

但是我们可以直接查看GO编译后的导出数据

```
go tool compile -S yourfile.go 查看汇编
go list -json
```

装饰器和元编程

`@register`, `@filter`

代码中的 `@register` 和 `@filter.command` 看起来很像 Java 的注解（Annotation），但它们是一回事吗？

```
@register("helloworld", "YourName", "一个简单的 Hello World 插件", "1.0.0")
class MyPlugin(Star):
    # ...

@filter.command("helloworld")
async def helloworld(self, event: AstrMessageEvent):
    # ...
```

Astrbot究竟是如何直到你写了一个叫 `helloworld` 的函数的？它怎么直到有人开发/helloworld的时候要找到这个函数？

答案就在 `@` 里

在 Python 里，这不叫“注解”，叫**装饰器 (Decorator)**。它不是给编译器看的标签，它是一个**捕兽夹**

在C里，“函数即变量”，函数指针是指向代码段的**内存地址**
而在pyhon里，“一切都是对象”